

DIPLOMA DATA ENGINEER PYTHON FOR DATA ENGINEERING

Docente: Angel Tintaya, Tech MBA



REGLAS



Se requiere **puntualidad** para un mejor desarrollo del curso.



Para una mayor concentración **mantener silenciado el micrófono** durante la sesión.



Las preguntas se realizarán **a través del chat** y en caso de que lo requieran **podrán activar el micrófono**.



Realizar las actividades y/o tareas encomendadas en **los plazos determinados**.



Identificarse en la sala Zoom con el primer nombre y primer apellido.



EVALUACIÓN

Asistencia (Curso):

mínimo 80% sesiones para
recibir la certificación

Participación
 (20%) + ***Caso 1***
 (10%) + ***Caso 2***
 (10%) + ***Exámen Final***
 (60%)



OBJETIVO

General:

- Aprender a utilizar el lenguaje Python para el desarrollo de procesos ETL.

Específicos:

- Dominar Python como lenguaje de programación.
- Entender y poner en práctica el desarrollo de procesos ETL utilizando Python.



Pandas - II



Filtrado, Ordenamiento y Selección

- Utilizaremos el Dataframe: df_base

(<https://raw.githubusercontent.com/AngelTintaya/datasets/main/coordenadas.tsv>)

```
df_base = pd.read_csv('https://raw.githubusercontent.com/AngelTintaya/datasets/main/coordenadas.tsv', sep='\t')
df_base.head(5)
```

	CODIGO	FUENTE	PROV	DPTO	DIST	X	Y
0	9700083	API	AREQUIPA	AREQUIPA	VITOR	-71.938900	-16.466000
1	9700813	API	AREQUIPA	AREQUIPA	AREQUIPA	-71.536221	-16.395467
2	9702087	API	TRUJILLO	LA LIBERTAD	TRUJILLO	-79.037315	-8.128069
3	9701858	API	AREQUIPA	AREQUIPA	CAYMA	-71.549208	-16.392485
4	9700022	API	AREQUIPA	AREQUIPA	SOCABAYA	-71.524635	-16.441262



Filtrado, Ordenamiento y Selección

- Filtrado: Un filtro
- Filtrado: Múltiples filtros
- Ordenamiento de datos en un Dataframe
- Selección de datos de un Dataframe
- Selección con loc
- Selección con iloc



Filtrado, Ordenamiento y Selección

Filtrado:

Nomenclatura: `df[<Filtro>]`

Filtrado: Un filtro

- DataFrame cuya fuente proviene de una API:

`<Filtro>`:

```
df_base['FUENTE'] == 'API'
```

```
df_base[df_base['FUENTE'] == 'API'].head(2)
```

	CODIGO	FUENTE	PROV	DPTO	DIST	X	Y
0	9700083	API	AREQUIPA	AREQUIPA	VITOR	-71.938900	-16.466000
1	9700813	API	AREQUIPA	AREQUIPA	AREQUIPA	-71.536221	-16.395467



Filtrado, Ordenamiento y Selección

Filtrado: Múltiples Filtros

- DataFrame cuya fuente proviene de una API y cuyo departamento no provenga de Arequipa ni de la Libertad:

<Filtro>:

```
(df_base['FUENTE']=='API') & ~(df_base['DPTO'].isin(['AREQUIPA', 'LA LIBERTAD']))
```

```
df_base[(df_base['FUENTE']=='API') & ~(df_base['DPTO'].isin(['AREQUIPA', 'LA LIBERTAD']))].head(2)
```

	CODIGO	FUENTE	PROV	DPTO	DIST	X	Y
545	9701029	API	SAN ROMAN	PUNO	JULIACA	-70.127531	-15.502608
577	9701699	API	SAN ROMAN	PUNO	JULIACA	-70.131694	-15.491775

Filtrado, Ordenamiento y Selección

Ordenamiento de datos de datos en un Dataframe

- Nomenclatura:

`df.sort_values(by=<list_columns_to_order>, ascending=<list_booleans>)`

```
df_base.sort_values(by=['FUENTE', 'PROV', 'CODIGO'], ascending=[True, False, True], inplace=True)
df_base.head()
```

	CODIGO	FUENTE	PROV	DPTO	DIST	X	Y
3199	9699985	API	TRUJILLO	LA LIBERTAD	TRUJILLO	-79.019552	-8.104854
7938	9699985	API	TRUJILLO	LA LIBERTAD	TRUJILLO	-79.021932	-8.113123
9434	9699985	API	TRUJILLO	LA LIBERTAD	TRUJILLO	-79.038636	-8.126615
11144	9699985	API	TRUJILLO	LA LIBERTAD	TRUJILLO	-79.040091	-8.100903
1991	9699986	API	TRUJILLO	LA LIBERTAD	TRUJILLO	-79.012899	-8.087046

Filtrado, Ordenamiento y Selección

Selección de datos en un Dataframe

- Nomenclatura:

`df[<lista_campos_a_seleccionar>]`

```
df_base[['CODIGO', 'FUENTE']][7:10]
```

	CODIGO	FUENTE
7	9701160	API
8	9700030	API
9	9701836	API



Filtrado, Ordenamiento y Selección

Selección loc

- Nomenclatura:

`df.loc[<index_start>:<index_stop>, <lista_de_campos>]`

`df.loc[<lista_indices>, <lista_de_campos>]`

```
df_base.loc[7:9, ['CODIGO', 'FUENTE']]
```

	CODIGO	FUENTE
7	9701160	API
8	9700030	API
9	9701836	API

```
df_base.loc[[3, 7, 9], ['CODIGO', 'FUENTE']]
```

	CODIGO	FUENTE
3	9701858	API
7	9701160	API
9	9701836	API



Filtrado, Ordenamiento y Selección

Selección iloc

- Nomenclatura:

`df.iloc[<idx_start>:<idx_stop> | <list_index>, <idx_col_start>:<idx_col_stop>]`

`df.iloc[<idx_start>:<idx_stop> | <list_index>, <Lista index col>]`

```
df_base.iloc[7:9, :2]
```

	CODIGO	FUENTE
7	9701160	API
8	9700030	API

```
df_base.iloc[7:9, [0,-2]]
```

	CODIGO	X
7	9701160	-79.035671
8	9700030	NaN



Uniendo y concatenando Dataframes

- Dataframes a utilizar:

```
df1 = pd.DataFrame({'A': ['A0', 'A1', 'A2'],
                    'B': ['B0', 'B1', 'B2'],
                    'C': ['C0', 'C1', 'C2'],
                    'D': ['D0', 'D1', 'D2']},
                    index=[0, 1, 2])
```

df1

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2



```
df2 = pd.DataFrame({'A': ['A3', 'A4', 'A5'],
                    'B': ['B3', 'B4', 'B5'],
                    'C': ['C3', 'C4', 'C5'],
                    'D': ['D3', 'D4', 'D5']},
                    index=[3, 4, 5])
```

df2

	A	B	C	D
3	A3	B3	C3	D3
4	A4	B4	C4	D4
5	A5	B5	C5	D5



Uniendo y concatenando Dataframes

- Dataframes a utilizar:

```
df3 = pd.DataFrame({'B': ['B2', 'B3', 'B7'],
                    'D': ['D2', 'D3', 'D7'],
                    'F': ['F2', 'F3', 'F7']},
                    index=[2, 3, 7])
```

df3

	B	D	F
2	B2	D2	F2
3	B3	D3	F3
7	B7	D7	F7



Uniendo y concatenando Dataframes

Concatenando DataFrames: `pd.concat()`

- Nomenclatura: `pd.concat(<Lista de Dataframes a concatenar>)`

```
pd.concat([df1,df2])
```

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3
4	A4	B4	C4	D4
5	A5	B5	C5	D5



```
pd.concat([df3,df1])
```

	B	D	F	A	C
2	B2	D2	F2	NaN	NaN
3	B3	D3	F3	NaN	NaN
7	B7	D7	F7	NaN	NaN
0	B0	D0	NaN	A0	C0
1	B1	D1	NaN	A1	C1
2	B2	D2	NaN	A2	C2



Uniendo y concatenando Dataframes

Concatenando DataFrames - Ordenando Columnas

- Nomenclatura:

`pd.concat(<list_dataframes>, sort=True)`

```
pd.concat([df3, df1], sort=True)
```



	A	B	C	D	F
2	NaN	B2	NaN	D2	F2
3	NaN	B3	NaN	D3	F3
7	NaN	B7	NaN	D7	F7
0	A0	B0	C0	D0	NaN
1	A1	B1	C1	D1	NaN
2	A2	B2	C2	D2	NaN

`pd.concat(<list_dataframes>).sort_index(axis=1)`

```
pd.concat([df3, df1]).sort_index(axis = 1)
```



	A	B	C	D	F
2	NaN	B2	NaN	D2	F2
3	NaN	B3	NaN	D3	F3
7	NaN	B7	NaN	D7	F7
0	A0	B0	C0	D0	NaN
1	A1	B1	C1	D1	NaN
2	A2	B2	C2	D2	NaN

Uniendo y concatenando Dataframes

Concatenando DataFrames - Ordenando Índices

- Nomenclatura:

`pd.concat(<list_dataframes>).sort_index(axis=0)`

```
pd.concat([df3,df1]).sort_index(axis = 0)
```

	B	D	F	A	C
0	B0	D0	NaN	A0	C0
1	B1	D1	NaN	A1	C1
2	B2	D2	F2	NaN	NaN
2	B2	D2	NaN	A2	C2
3	B3	D3	F3	NaN	NaN
7	B7	D7	F7	NaN	NaN



Uniendo y concatenando Dataframes

Concatenando DataFrames - Reseteando Índices

- Nomenclatura: `df.reset_index(drop=True)`

```
pd.concat([df3,df1], sort=True).reset_index(drop=True)
```

	A	B	C	D	F
0	NaN	B2	NaN	D2	F2
1	NaN	B3	NaN	D3	F3
2	NaN	B7	NaN	D7	F7
3	A0	B0	C0	D0	NaN
4	A1	B1	C1	D1	NaN
5	A2	B2	C2	D2	NaN



Joins

- Unión con llave en común
- Merge con llave distinta
- Unión con múltiples llaves
- Left / Right / Inner / Outer Join



Joins

- Utilizaremos el Dataframe: df_base

(<https://raw.githubusercontent.com/AngelTintaya/datasets/main/coordenadas.tsv>)

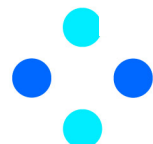
```
df_base = pd.read_csv('https://raw.githubusercontent.com/AngelTintaya/datasets/main/coordenadas.tsv', sep='\t')
df_base.head(5)
```

```
df_desfuente = pd.DataFrame({'FUENTE': ['API', 'WBS'],
                             'FNT': ['API', 'WBS'],
0      'DESFUENTE': ['API de Google', 'Webscrapimng a Google Maps'],
1      'ORIGEN': ['UM_COORD_EMPRESA', 'WEB PAGE']
2      })
```

```
df_desfuente.head()
```

```
3
4
```

	FUENTE	FNT	DESFUENTE	ORIGEN
0	API	API	API de Google	UM_COORD_EMPRESA
1	WBS	WBS	Webscrapimng a Google Maps	WEB PAGE



Joins

- Utilizaremos el Dataframe: df_desfuelle

```
df_desfuelle = pd.DataFrame({'FUENTE': ['API', 'WBS'],
                             'FNT': ['API', 'WBS'],
                             'DESFUENTE': ['API de Google', 'Webscrapimng a Google Maps'],
                             'ORIGEN': ['UM_COORD_EMPRESA', 'WEB PAGE']
                             })

df_desfuelle.head()
```

	FUENTE	FNT	DESFUENTE	ORIGEN
0	API	API	API de Google	UM_COORD_EMPRESA
1	WBS	WBS	Webscrapimng a Google Maps	WEB PAGE



Joins

Cruce utilizando un campo en común

- Nomenclatura:

`pd.merge(df_1, df_2, on=<campo_común>, how=<left | right | inner>)`

```
df_cruce = pd.merge(df_base,
                    df_desfuente[['FUENTE', 'DESFUENTE']],
                    on = 'FUENTE',
                    how = 'left' # right, inner
                    )

df_cruce.head(2)
```

	CODIGO	FUENTE	PROV	DPTO	DIST	X	Y	DESFUENTE
0	9700083	API	AREQUIPA	AREQUIPA	VITOR	-71.938900	-16.466000	API de Google
1	9700813	API	AREQUIPA	AREQUIPA	AREQUIPA	-71.536221	-16.395467	API de Google



Joins

Cruce utilizando un campo en común

- Nomenclatura:

`pd.merge(df_1, df_2, left_on=<campo_1>, right_on=<campo_2>, how=<left | right | inner>)`

```
df_cruce = pd.merge(df_base,
                    df_desfuente[['FNT', 'DESFUENTE']],
                    left_on = ['FUENTE'],
                    right_on = ['FNT'],
                    how = 'inner'
                    )

df_cruce.head(2)
```

	CODIGO	FUENTE	PROV	DPTO	DIST	X	Y	FNT	DESFUENTE
0	9700083	API	AREQUIPA	AREQUIPA	VITOR	-71.938900	-16.466000	API	API de Google
1	9700813	API	AREQUIPA	AREQUIPA	AREQUIPA	-71.536221	-16.395467	API	API de Google



Bonus: Borrando columnas en Dataframe

Método .drop()

- Nomenclatura:

`df.drop(<col_to_delete> | <list_columns_to_drop>, inplace=True, axis=1)`

```
df_cruce.drop('FNT', inplace=True, axis=1)
df_cruce.drop(['X', 'Y'], inplace=True, axis=1)
df_cruce.head(2)
```

	CODIGO	FUENTE	PROV	DPTO	DIST	DESFUENTE
0	9700083	API	AREQUIPA	AREQUIPA	VITOR	API de Google
1	9700813	API	AREQUIPA	AREQUIPA	AREQUIPA	API de Google



Bonus: Borrando columnas en Dataframe

Funcionalidad del

- Nomenclatura:
del df[<col_to_delete>]

```
del df_cruce['PROV']  
df_cruce.head(2)
```

	CODIGO	FUENTE	DPTO	DIST	DESFUENTE
0	9700083	API	AREQUIPA	VITOR	API de Google
1	9700813	API	AREQUIPA	AREQUIPA	API de Google



Bonus: Borrando columnas en Dataframe

Método .pop()

- Nomenclatura:
df.pop(<col_to_delete>)

```
df_cruce.pop('DPT0')
```

```
0      AREQUIPA
1      AREQUIPA
2  LA  LIBERTAD
3      AREQUIPA
4      AREQUIPA
```

```
...
```

```
14912      NaN
14913      NaN
14914      NaN
14915      NaN
14916      NaN
```

```
Name: DPT0, Length: 14917, dtype: object
```

```
df_cruce.head(2)
```

	CODIGO	FUENTE	DIST	DESFUENTE
0	9700083	API	VITOR	API de Google
1	9700813	API	AREQUIPA	API de Google



Trabajando con campos

- Creando un campo
- Rellenando Nulos
- Cambiando tipos de datos
- Funciones Lambda
- Interactuando con fechas
- Renombrado de columnas
- Mapeo de datos



Trabajando con campos

Dataframe: dict_clase

```
dict_clase = {'codigo': [9876654, 7665498, 6598764, 5498766, 8976654],
              'edad': [26, np.nan, 30, 42, 38],
              'ingreso': [2500, 2500, 3500, 4800, 4100],
              'estado_civil': ['Soltero', 'Soltero', 'Casado', 'Casado', 'Soltero'],
              'fecmatricula': ['25/07/2022', '21/07/2022', '28/07/2022', '26/07/2022', '22/07/2022']}

df_clase = pd.DataFrame(dict_clase)
df_clase
```

	codigo	edad	ingreso	estado_civil	fecmatricula
0	9876654	26.0	2500	Soltero	25/07/2022
1	7665498	NaN	2500	Soltero	21/07/2022
2	6598764	30.0	3500	Casado	28/07/2022
3	5498766	42.0	4800	Casado	26/07/2022
4	8976654	38.0	4100	Soltero	22/07/2022



Trabajando con campos

Creando un campo

- Nomenclatura: `df[<nuevo_campo>] = <Valor | Serie del Dataframe>`

```
df_clase['anho'] = '2022'
df_clase.head(2)
```

	codigo	edad	ingreso	estado_civil	fecmatricula	anho
0	9876654	26.0	2500	Soltero	25/07/2022	2022
1	7665498	NaN	2500	Soltero	21/07/2022	2022

```
df_clase['ingreso_2'] = df_clase['ingreso']*2
df_clase.head(2)
```

	codigo	edad	ingreso	estado_civil	fecmatricula	anho	ingreso_2
0	9876654	26.0	2500	Soltero	25/07/2022	2022	5000
1	7665498	NaN	2500	Soltero	21/07/2022	2022	5000



Trabajando con campos

Rellenando Nulos: Método .fillna()

- Nomenclatura: <Series | Dataframe>.fillna(<Value_to_fill_nan>)

```
df_clase['edad'] = df_clase['edad'].fillna(df_clase.edad.mean())
df_clase.head()
```

	codigo	edad	ingreso	estado_civil	fecmatricula
0	9876654	26.0	2500	Soltero	25/07/2022
1	7665498	34.0	2500	Soltero	21/07/2022
2	6598764	30.0	3500	Casado	28/07/2022
3	5498766	42.0	4800	Casado	26/07/2022
4	8976654	38.0	4100	Soltero	22/07/2022

```
df_clase.edad.mean()
34.0
```



Trabajando con campos

Cambiando tipo de datos

- Nomenclatura: <Serie | Dataframe>.astype(<new_data_type>)
- Cuando se transforma a un tipo de dato string, es común utilizarlo junto al método str en una serie para vectorizar funciones string.
Nomenclatura: <Serie>.astype("str").str.<método_string>

```
df_clase['DNI'] = df_clase['codigo'].astype('str').str.zfill(8)
df_clase.head(2)
```

	codigo	edad	ingreso	estado_civil	fecmatricula	DNI
0	9876654	26.0	2500	Soltero	25/07/2022	09876654
1	7665498	NaN	2500	Soltero	21/07/2022	07665498



Trabajando con campos

Funciones Lambda: Aplicado a una serie (Un campo)

- Nomenclatura: <Serie>.apply(lambda <variable>: <function>)

```
df_clase['TIP_INGRESO'] = df_clase['ingreso'].apply(lambda x: 'ALTO' if x > 3000 else 'BAJO')
df_clase.head(3)
```

	codigo	edad	ingreso	estado_civil	fecmatricula	DNI	TIP_INGRESO
0	9876654	26.0	2500	Soltero	25/07/2022	09876654	BAJO
1	7665498	NaN	2500	Soltero	21/07/2022	07665498	BAJO
2	6598764	30.0	3500	Casado	28/07/2022	06598764	ALTO

```
df_clase['CODMESMATRICULA'] = df_clase['fecmatricula'].apply(lambda x: x.split('/')[2]+x.split('/')[1])
df_clase.head(1)
```

	codigo	edad	ingreso	estado_civil	fecmatricula	DNI	TIP_INGRESO	CODMESMATRICULA
0	9876654	26.0	2500	Soltero	25/07/2022	09876654	BAJO	202207

Trabajando con campos

Funciones Lambda: Aplicado a un Dataframe (Múltiples campos)

- Nomenclatura:

<Dataframe>.apply(lambda <series_variable>: <function>, axis=1)

```
df_clase['TIP_INGRESO2'] = df_clase[['ingreso', 'edad']].apply(lambda x: 'ALTO' if x['ingreso'] > 4000 else 'MEDIO' if x['ingreso'] > 2000 and x['edad'] < 30 else 'BAJO', axis = 1)
df_clase.head(2)
```

	codigo	edad	ingreso	estado_civil	fecmatricula	DNI	TIP_INGRESO	CODMESMATRICULA	TIP_INGRESO2
0	9876654	26.0	2500	Soltero	25/07/2022	09876654	BAJO	202207	MEDIO
1	7665498	NaN	2500	Soltero	21/07/2022	07665498	BAJO	202207	BAJO

Nota: axis=1 significa que se replicará a cada índice de fila

Trabajando con campos

Funciones Lambda: Utilizando una función definida por el usuario

```
def genera_tipingreso(ingreso, edad):
    if ingreso > 4000:
        result = 'ALTO'
    elif ingreso > 2000 and edad < 30:
        result = 'MEDIO'
    else:
        result = 'BAJO'
    return result
```

```
df_clase['TIP_INGRESO3'] = df_clase[['ingreso', 'edad']].apply(lambda x: genera_tipingreso(x['ingreso'], x['edad']), axis=1)
df_clase.head(2)
```

	codigo	edad	ingreso	estado_civil	fecmatricula	DNI	TIP_INGRESO	CODMESMATRICULA	TIP_INGRESO2	TIP_INGRESO3
0	9876654	26.0	2500	Soltero	25/07/2022	09876654	BAJO	202207	MEDIO	MEDIO
1	7665498	NaN	2500	Soltero	21/07/2022	07665498	BAJO	202207	BAJO	BAJO

Trabajando con campos

Interactuando con Fechas

- Es muy común utilizar las funciones de datetime dentro de la función lambda, para ello es requisito importar la librería datetime.

```
from datetime import datetime
```

```
df_clase['DATETIME'] = df_clase['fecmatricula'].apply(lambda x: datetime.strptime(x, "%d/%m/%Y"))
df_clase.iloc[:2, [0,1,2,3,4,7,-1]]
```

	codigo	edad	ingreso	estado_civil	fecmatricula	CODMESMATRICULA	DATETIME
0	9876654	26.0	2500	Soltero	25/07/2022	202207	2022-07-25
1	7665498	NaN	2500	Soltero	21/07/2022	202207	2022-07-21



Trabajando con campos

Interactuando con Fechas

Sin datetime

```
df_clase['CODMESMATRICULA'] = df_clase['fecmatricula'].apply(lambda x: x.split('/')[2]+x.split('/')[1])
df_clase.head(1)
```

	codigo	edad	ingreso	estado_civil	fecmatricula	DNI	TIP_INGRESO	CODMESMATRICULA	
0	9876654	26.0	2500	Soltero	25/07/2022	09876654	BAJO	202207	

Con datetime

```
df_clase['CODMES'] = df_clase[
    'fecmatricula'].apply(lambda x: datetime.strptime(x, "%d/%m/%Y").strftime('%Y%m'))
df_clase.iloc[:2, [0,1,2,3,4,7,-2,-1]]
```

	codigo	edad	ingreso	estado_civil	fecmatricula	CODMESMATRICULA	DATETIME	CODMES	
0	9876654	26.0	2500	Soltero	25/07/2022	202207	2022-07-25	202207	
1	7665498	NaN	2500	Soltero	21/07/2022	202207	2022-07-21	202207	

Trabajando con campos

Mapeando campos: Método .map()

- Nomenclatura: <Serie>.map(dict_to_map)
- Genera valores de acuerdo a una asignación de entrada (Diccionario).

```
df_clase['DETINGRESO'] = df_clase['TIP_INGRESO'].map({
    'BAJO': 'Ingreso bajo',
    'MEDIO': 'Ingreso medio',
    'ALTO': 'Ingreso alto',
})
df_clase.iloc[[0,2], [0,1,2,3,4,6,-3,-2,-1]]
```

	codigo	edad	ingreso	estado_civil	fecmatricula	TIP_INGRESO	DATETIME	CODMES	DETINGRESO
0	9876654	26.0	2500	Soltero	25/07/2022	BAJO	2022-07-25	202207	Ingreso bajo
2	6598764	30.0	3500	Casado	28/07/2022	ALTO	2022-07-28	202207	Ingreso alto

Guardado de un dataframe

Guardado en CSV

```
df_clase.to_csv('/content/drive/MyDrive/Datasets/dataframes/df_clase.csv', index=False)
```

Guardado en JSON

```
df_clase.to_json('/content/drive/MyDrive/Datasets/dataframes/df_clase.json', orient='records', lines=True)
```

Guardado en Base de Datos

```
from sqlalchemy import create_engine

engine = create_engine('sqlite:///content/drive/MyDrive/Datasets/dataframes/my_database.db')
df_clase.to_sql('df_clase', con=engine, index=False, if_exists='replace')
```

Caso de Uso: RetailMax

En RetailMax, el área de inteligencia comercial busca entender cómo varían las ventas según el tipo de cliente y la categoría de productos. Para lograrlo, se dispone de tres fuentes principales de información:

- Ventas: Contiene el detalle de cada transacción.
- Clientes: Incluye información demográfica y de segmentación de los clientes.
- Productos: Clasifica los artículos en diferentes categorías.

El objetivo es unificar estas fuentes para poder segmentar las ventas de manera cruzada: Por tipo de cliente y categoría de producto.

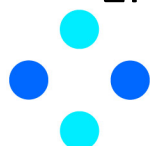
Trabajo



Trabajo: Agrupaciones y Ordenamiento

- Obtener el máximo y mínimo código (CODIGO) y total de departamentos (DPTO) de cada fuente (FUENTE)
- Nombrarlos como MIN_CODIGO, MAX_CODIGO y COUNT_DPTO
- Ordenarlos por MIN_CODIGO y MAX_CODIGO de manera ascendente y COUNT_DPTO de manera descendente.
- Utilizar el siguiente dataset: <https://raw.githubusercontent.com/AngelTintaya/datasets/main/coordenadas.tsv>
- OBS: Será necesario lidiar con los missing values de DPTO, para ello reemplacemos estos valores por un “-”
- El objetivo es obtener el siguiente resultado:

	FUENTE	MIN_CODIGO	MAX_CODIGO	COUNT_DPTO
0	WBS	9699985	9703230	1512
1	API	9699985	9703237	13405



¡GRACIAS!

